

Universidad de La Habana
Facultad de Matemática y Computación



Sistema web para el autorreporte de Eventos Supuestamente Atribuibles a la Vacunación o Inmunización (ESAVI) en el Instituto Finlay de Vacunas (IFV)

Autor:

Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos

MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

15 de junio de 2026

github.com/JabelResendiz/thesis

Capítulo 1

Validación de la aplicación

En el Capítulo ?? se describieron los detalles de implementación del sistema: la arquitectura, las tecnologías empleadas y las decisiones de diseño. Sin embargo, la implementación por sí sola no garantiza que el sistema cumpla con los requerimientos definidos ni que ofrezca una experiencia adecuada a sus usuarios. Este capítulo somete el producto a un proceso de validación que abarca cuatro dimensiones complementarias: corrección funcional, rendimiento bajo carga, calidad del despliegue web y usabilidad percibida.

1.1. Propósito de la Validación

La validación del sistema persigue demostrar cuatro afirmaciones concretas:

1. **Los flujos principales operan correctamente.** El registro de reportes, la asignación a revisores, la gestión de revisiones y las notificaciones automáticas funcionan conforme a los requerimientos funcionales definidos en el Capítulo ??.
2. **El sistema soporta la concurrencia esperada.** Los endpoints críticos —creación y búsqueda de reportes— responden bajo carga concurrente sin superar los umbrales de tiempo definidos, incluso con volúmenes de datos acumulados.
3. **La calidad web cumple con los estándares.** El frontend desplegado satisface los criterios de rendimiento, accesibilidad, mejores prácticas y SEO evaluados mediante Lighthouse.
4. **Los usuarios finales consideran la interfaz útil y fácil de usar.** La evaluación mediante el cuestionario SUS (System Usability Scale) debe arrojar una puntuación superior a 68, umbral que define la usabilidad aceptable según Brooke [brooke1996sus].

1.2. Enfoques de Validación

Para cubrir las cuatro afirmaciones anteriores, se aplicaron los siguientes enfoques complementarios:

- **Pruebas funcionales:** ejecución de los flujos principales del sistema, verificando que cada paso produce el resultado esperado según los requerimientos. Las características del entorno de ejecución se detallan en cada caso de prueba.
- **Pruebas de rendimiento:** evaluación de la capacidad del sistema bajo carga concurrente mediante k6, con el backend y el cliente en máquinas independientes conectadas por red hotspot. Las especificaciones del hardware se presentan en la Sección 1.3.
- **Validación de calidad web:** auditoría del frontend desplegado en Vercel mediante PageSpeed Insights (Lighthouse), evaluando rendimiento, accesibilidad, mejores prácticas y SEO.
- **Pruebas de usabilidad:** recorrido cognitivo con 5 participantes representativos de los cuatro perfiles del sistema, complementado con el cuestionario SUS.

1.3. Entorno de Pruebas de Rendimiento

Las pruebas de rendimiento se ejecutaron con k6, herramienta de código abierto para pruebas de carga. El backend se desplegó en una máquina con sistema operativo Linux Ubuntu 22.04, procesador Intel Core i5 de 4 núcleos a 2.5 GHz, 8 GB de RAM y almacenamiento SSD de 256 GB. k6 se ejecutó en una máquina cliente independiente con Windows 11, procesador Intel Core i7 de 8 núcleos a 3.0 GHz y 16 GB de RAM, conectadas mediante red hotspot a 2.4 GHz. Esta separación garantiza que las mediciones no se vean afectadas por competencia de recursos entre el cliente de pruebas y el servidor.

Se emplearon datos simulados que representan solicitudes realistas de creación y búsqueda de reportes ESAVI, con una carga útil equivalente a la del sistema en producción. Las métricas analizadas fueron: latencia promedio y percentil 95, throughput y tasa de solicitudes exitosas. Se definió como umbral de tiempo aceptable 3000 ms y como criterio de fallo funcional cualquier código de estado distinto de 2XX.

1.4. Pruebas funcionales

El objetivo de estas pruebas fue validar que las funcionalidades críticas del sistema ESAVI cumplen con los requisitos de diseño y garantizan el flujo de información

para la farmacovigilancia [3, 4]. Para ello, se empleó una estrategia de caja negra simulando operaciones de usuarios normales desde el navegador, utilizando datos sintéticos (*mock*) para verificar la persistencia y la respuesta del sistema ante entradas representativas [5].

El estado de cumplimiento de las funciones principales se resume en la Tabla 1.1, donde se evidencia que todos los casos de prueba seleccionados fueron ejecutados con éxito, confirmando la operatividad de la plataforma [1].

1.4.1. Evidencias de ejecución

En esta sección se presentan capturas de pantalla que evidencian la ejecución correcta de los principales flujos del sistema, validando la interacción entre el usuario y la interfaz, así como la respuesta del servidor en tiempo real.

Cada flujo funcional se documenta de manera secuencial, permitiendo demostrar de forma estructurada el comportamiento integral del sistema de extremo a extremo.

Flujo completo del manejo de reportes ESAVI

Este flujo describe el ciclo completo de gestión de un reporte ESAVI, desde su creación por el usuario hasta su evaluación final por el médico evaluador y la consulta del estado actualizado del caso, como se muestra en la Figura 1.1.

Tabla 1.1: Resumen de casos de prueba funcionales

ID	Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido	¿Pasó?
CP01	Inicio de sesión	Credenciales válidas	Acceso al sistema según el rol del usuario	Acceso concedido correctamente	✓
CP02	Registro de reporte ESAVI	Datos completos del reporte	Reporte almacenado y documento PDF generado	Operación realizada correctamente	✓
CP03	Consulta por número de notificación	Número de notificación existente	Visualización del estado y detalles del reporte	Información mostrada correctamente	✓
CP04	Gestión de usuarios	Datos de un nuevo médico o jefe de sección	Usuario registrado y disponible para administración	Usuario creado correctamente	✓
CP05	Asignación de reporte a médico	Reporte pendiente y médico disponible	Asignación creada y visible para el médico	Asignación realizada correctamente	✓
CP06	Resolución de reportes duplicados	Reportes identificados como posibles duplicados	Actualización del estado según la decisión tomada	Proceso ejecutado correctamente	✓
CP07	Visualización del dashboard administrativo	Solicitud de acceso al panel de indicadores	Presentación de estadísticas e indicadores	Dashboard mostrado correctamente	✓
CP08	Filtrado de reportes	Parámetros de búsqueda y filtrado	Listado de reportes coincidentes	Resultados obtenidos correctamente	✓
CP09	Revisión médica del reporte	Reporte asignado al médico	Actualización del estado y registro de la revisión	Revisión completada correctamente	✓
CP10	Inicio de sesión con credenciales inválidas	Usuario o contraseña incorrectos	Rechazo del acceso y mensaje de error	Validación ejecutada correctamente	✓
CP11	Registro de reporte con datos obligatorios ausentes	Formulario incompleto	Notificación de errores de validación	Validación ejecutada correctamente	✓
CP12	Consulta de número de notificación inexistente	Número no registrado	Notificación de reporte no encontrado	Respuesta emitida correctamente	✓

(a) Creación de reporte ESAVI

(b) Confirmación de registro del reporte

(c) Asignación del reporte por el jefe de vacunación municipal a un médico evaluador

(d) Visualización del reporte por el médico evaluador asignado

(e) Actualización del estado del reporte tras la evaluación médica

Figura 1.1: Flujo del sistema ESAVI para la creación, asignación y evaluación de reportes de eventos adversos posteriores a la vacunación

Gestión de acceso y registro de médico evaluador

Este flujo abarca el proceso de registro de un médico evaluador en el sistema y su posterior autenticación mediante inicio de sesión, garantizando el acceso controlado a las funcionalidades del módulo de revisión de reportes.

1.4.2. Análisis de resultados

1.5. Pruebas de rendimiento

1.5.1. Entorno de Pruebas

Las pruebas de rendimiento se ejecutaron con k6, herramienta de código abierto para pruebas de carga. El backend se desplegó en una máquina con sistema operativo Linux Ubuntu 22.04, procesador Intel Core i5 de 4 núcleos a 2.5 GHz, 8 GB de RAM y almacenamiento SSD de 256 GB. k6 se ejecutó en una máquina cliente independiente con Windows 11, procesador Intel Core i7 de 8 núcleos a 3.0 GHz y 16 GB de RAM, conectadas mediante red hotspot a 2.4 GHz. Esta separación garantiza que las mediciones no se vean afectadas por competencia de recursos entre el cliente de pruebas y el servidor.

Se emplearon datos simulados que representan solicitudes realistas de creación y búsqueda de reportes ESAVI, con una carga útil equivalente a la del sistema en producción. Las métricas analizadas fueron: latencia promedio y percentil 95 (`http_req_duration`), throughput (`http_reqs`) y tasa de solicitudes exitosas (status 201). Se definió como umbral de tiempo aceptable 3000 ms y como criterio de fallo funcional cualquier código de estado distinto de 2XX.

El objetivo fue identificar, para cada endpoint crítico, el número máximo de usuarios concurrentes que el sistema soporta antes de superar estos umbrales.

Creación de Reporte de ESAVI

Para evaluar el rendimiento del endpoint de creación de reportes se diseñó una prueba de carga incremental con datos simulados. Cada solicitud contiene un reporte completo con 13 vacunas y 10 lotes por vacuna. Los detalles de configuración —etapas de rampa (1 min subida, 3 min estable, 1 min bajada), generación de datos y código k6— se presentan en el Anexo A.1. Las pruebas se ejecutaron de forma independiente para 5, 10, 30, 50, 70, 80, 100, 150 y 200 VUs.

Las Figuras 1.2, 1.3 y 1.4 muestran la evolución de las métricas conforme aumenta la carga concurrente.

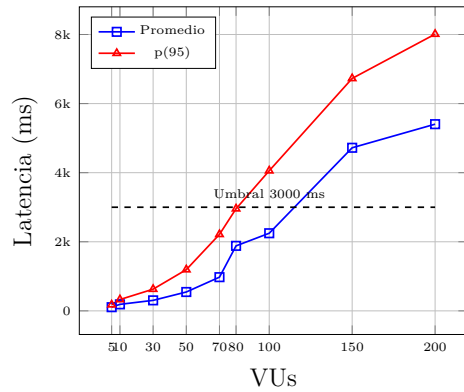


Figura 1.2: Evolución de la latencia según usuarios concurrentes.

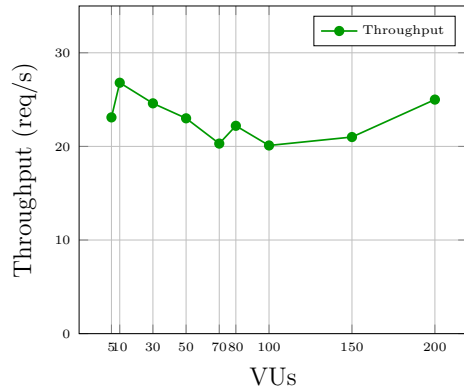


Figura 1.3: Evolución del throughput según usuarios concurrentes.

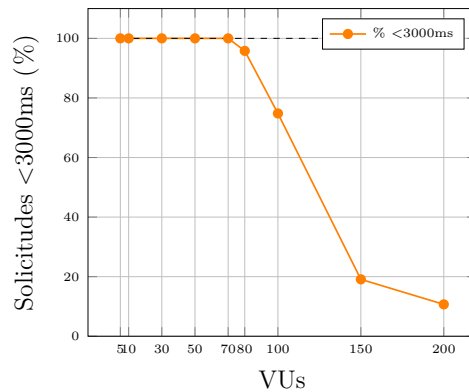


Figura 1.4: Porcentaje de solicitudes por debajo del umbral de 3000 ms.

Los resultados muestran tres zonas de comportamiento. Hasta 70 VUs, la latencia se mantiene por debajo de 1000 ms con el 100% de las solicitudes respondiendo en menos de 3000 ms. A partir de 80 VUs aparece el primer punto de degradación: la latencia asciende a 1882 ms. Con 100 VUs alcanza 2246 ms, y con 200 VUs supera los 5400 ms.

El throughput se mantuvo estable entre 20 y 26 req/s en todos los niveles de carga. Esto indica que el sistema alcanzó su capacidad máxima de procesamiento sin degradar la tasa de solicitudes exitosas: los usuarios adicionales no aumentan el rendimiento, sino que se traducen en mayor latencia.

Hasta 70 VUs, el 100% de las solicitudes responde en menos de 3000 ms. A partir de 80 VUs este porcentaje comienza a caer: 95.8% con 80 VUs, 74.8% con 100 VUs, y se desploma al 19.1% y 10.7% con 150 y 200 VUs respectivamente. En ningún momento se registraron errores funcionales (100% status 201), lo que confirma que la degradación es atribuible a las limitaciones del entorno de pruebas y no a deficiencias del código.

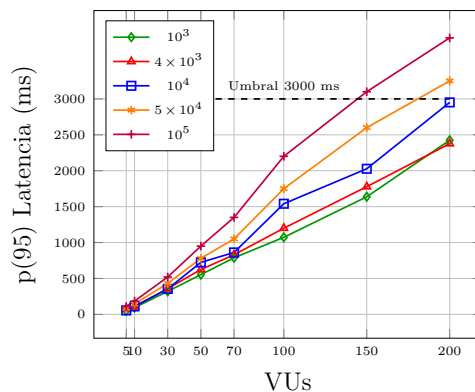
La prueba de carga sobre el endpoint de creación de reportes permite concluir que el sistema ofrece un rendimiento sólido bajo concurrencia moderada, con una zona de operación óptima donde la experiencia de usuario se mantiene fluida y sin degradación. La prueba simula un escenario más exigente que el uso real —cada usuario virtual envía solicitudes de forma ininterrumpida—, por lo que los resultados representan una cota inferior conservadora de la capacidad real del sistema.

Al superar el umbral de operación óptima, la latencia comienza a degradarse progresivamente, aunque sin comprometer la funcionalidad: en ningún nivel de carga se registraron errores, lo que confirma que las limitaciones observadas son atribuibles al entorno de pruebas y no a deficiencias del código desarrollado.

Búsqueda de Reporte por Número de Notificación

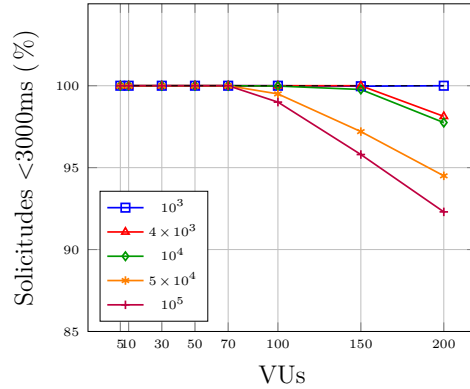
Para evaluar el rendimiento del endpoint de búsqueda se diseñó una prueba de carga incremental variando dos factores: el tamaño de la base de datos y la concurrencia. Se pobló la base de datos con 1,000, 10,000, 50,000 y 100,000 reportes, y para cada nivel se ejecutaron pruebas independientes con 5, 10, 30, 50 y 100 VUs. Cada solicitud busca un número de notificación aleatorio extraído de un archivo CSV con los identificadores reales de los reportes almacenados. Los detalles de configuración, generación de datos y código k6 se presentan en el Anexo A.2.

Las Figuras 1.5 y 1.6 muestran la evolución de las métricas conforme aumenta la carga concurrente para cada tamaño de base de datos.



Hasta 70 VUs, todas las curvas se mantienen agrupadas por debajo de 1000 ms, sin diferencias significativas entre tamaños de base de datos. A partir de 100 VUs las curvas comienzan a separarse: 10^3 , 4×10^3 y 10^4 reportes se mantienen por debajo del umbral de 3000 ms incluso con 200 VUs, mientras que 5×10^4 lo supera con 200 VUs (3250 ms) y 10^5 lo hace desde los 150 VUs (3100 ms). Esto evidencia que el volumen de datos impacta progresivamente en la latencia bajo alta concurrencia.

Figura 1.5: Evolución del p(95) de latencia según usuarios concurrentes para 1,000, 4,000, 10,000, 50,000 y 100,000 reportes.



Con 10^3 , 4×10^3 y 10^4 reportes, más del 97% de las solicitudes responde por debajo de 3000 ms incluso con 200 VUs. Con 5×10^4 y 10^5 reportes, el porcentaje descende al 94.5% y 92.3% respectivamente con 200 VUs, manteniéndose siempre por encima del 90%. En ningún nivel se registraron errores funcionales (100% status 202).

Figura 1.6: Porcentaje de solicitudes por debajo del umbral de 3000 ms según usuarios concurrentes y tamaño de base de datos.

Los resultados confirman que la búsqueda por número de notificación es robusta tanto frente a la concurrencia como al crecimiento del volumen de datos. En todos los niveles evaluados —hasta 200 VUs y 10^5 reportes— el sistema mantuvo una tasa de éxito del 100% (status 202), sin registrar errores funcionales.

Incluso en las condiciones más exigentes, más del 90% de las solicitudes respondió por debajo del umbral de 3000 ms, lo que evidencia que la búsqueda está fortalecida para operar bajo alta concurrencia con grandes volúmenes de reportes acumulados. La degradación observada es progresiva y predecible, sin caídas abruptas, lo que permite anticipar el comportamiento del sistema ante escenarios de crecimiento futuro.

Escenario Combinado

Para validar el comportamiento del sistema bajo una carga de trabajo realista con múltiples endpoints conviviendo, se diseñó un escenario con dos perfiles de tráfico simultáneos: tráfico público (85 VUs, alternando 65% creación y 35% búsqueda, con pausas de 1 a 3 segundos) y tráfico de jefes municipales (15 VUs, consultando el dashboard con autenticación Bearer rotando entre tokens reales, con pausas de 2 a 6 segundos). Las métricas se aislaron mediante tags, permitiendo analizar cada tipo de solicitud por separado. Los detalles del script se presentan en el Anexo A.3.

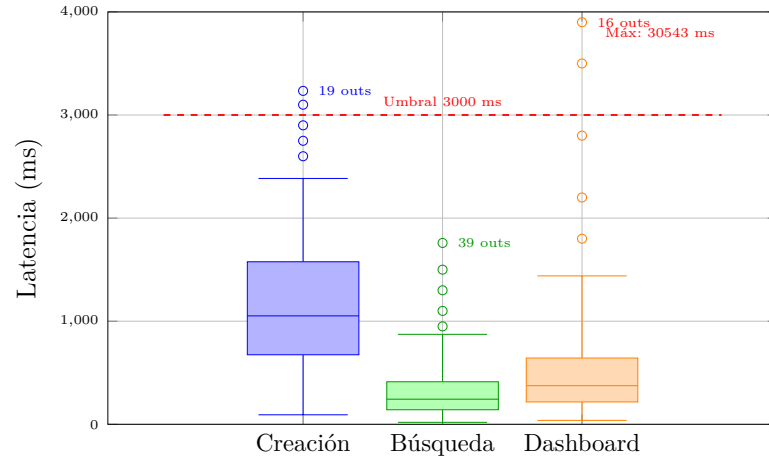


Figura 1.7: Distribución de latencias por tipo de endpoint. Las cajas representan el 50 % central de los datos; los puntos sobre el bigote superior son valores atípicos.

La Figura 1.7 permite apreciar visualmente la dispersión de cada endpoint. La búsqueda presenta la caja más compacta y cercana a cero, reflejando su menor variabilidad. La creación muestra la mayor dispersión, con el 50 % central de las solicitudes entre 674.4 ms y 1576.6 ms. El dashboard, aunque más concentrado que la creación, registra un valor atípico extremo de 30543 ms, apenas visible fuera de escala.

Tabla 1.2: Métricas detalladas del escenario combinado con 100 VUs.

Métrica	Creación	Búsqueda	Dashboard
Solicitudes	1190	643	203
Promedio (ms)	1176.7	319.2	780.6
Mediana (ms)	1051.6	243.6	374.6
Q1 (ms)	674.4	141.1	216.7
Q3 (ms)	1576.6	412.7	642.8
p(95) (ms)	2384.6	872.7	1439.6
Mínimo (ms)	92.2	19.2	38.0
Máximo (ms)	3234.1	1759.0	30543.1
Outliers	19	39	16
% éxito	100 %	100 %	100 %

La Tabla 1.2 complementa el análisis con los valores precisos. Se confirma que los tres endpoints mantuvieron el 100 % de solicitudes exitosas y que la mediana de cada uno se sitúa por debajo de los 1100 ms. La creación de reportes presenta la mayor latencia, lo cual es esperable por tratarse de una operación de escritura. Los outliers detectados —19, 39 y 16 respectivamente— son casos aislados que no afectan la experiencia general. En conjunto, el sistema demuestra estar preparado para operar

bajo condiciones normales de uso con múltiples perfiles de usuario concurrentes, sin interferencia entre endpoints públicos y protegidos.

1.6. Validación del Entorno de Producción y Calidad Web

Esta validación verifica la calidad del sistema una vez desplegado en un entorno real, considerando la experiencia del usuario final bajo distintas condiciones de conectividad. Para ello se empleó PageSpeed Insights, herramienta de Google que ejecuta el motor de auditoría Lighthouse y evalúa la calidad web con base en las métricas definidas en Web Vitals [1]. El sistema, desplegado en Vercel, se auditó simulando dos escenarios: un Moto G Power con red 4G lenta (móvil) y una estación de trabajo con conexión de alta velocidad (escritorio).

A continuación, se presentan y discuten los resultados para cada categoría evaluada.

1.6.1. Rendimiento

La categoría de rendimiento mide la velocidad de carga y la capacidad de respuesta de la página. Para ello, Lighthouse emplea las siguientes métricas:

- **Primer Despliegue de Contenido (FCP, por sus siglas en inglés):** tiempo que transcurre hasta que el navegador muestra el primer elemento visual de la página.
- **Despliegue del Contenido Más Grande (LCP):** tiempo necesario para renderizar el elemento de mayor tamaño visible para el usuario, siendo el principal indicador de velocidad de carga percibida.
- **Tiempo Total de Bloqueo (TBT):** tiempo durante el cual el hilo principal del navegador permanece bloqueado e incapaz de responder a las interacciones del usuario.
- **Índice de Velocidad (SI):** mide la rapidez con la que el contenido visible de la página se muestra progresivamente.
- **Desplazamiento Acumulado del Diseño (CLS):** cuantifica la estabilidad visual de la interfaz, reflejando los desplazamientos inesperados de los elementos durante la carga.

Las Tablas 1.3 y 1.4 muestran los resultados obtenidos para las páginas representativas del sistema.

Tabla 1.3: Resultados de rendimiento en entorno de escritorio.

Página	Puntaje	FCP	LCP	TBT	SI	CLS
Página principal	100	0.4 s	0.6 s	0 ms	0.7 s	0.00
Registro de ESAVI	100	0.4 s	0.6 s	0 ms	0.6 s	0.00
Panel informativo	100	0.4 s	0.6 s	0 ms	0.5 s	0.00

Tabla 1.4: Resultados de rendimiento en entorno móvil.

Página	Puntaje	FCP	LCP	TBT	SI	CLS
Página principal	90	2.2 s	2.9 s	0 ms	4.6 s	0.00
Registro de ESAVI	96	1.7 s	2.7 s	0 ms	1.7 s	0.00
Panel informativo	95	1.7 s	2.4 s	0 ms	3.9 s	0.00

Los resultados en escritorio reflejan un rendimiento excepcional: las tres páginas alcanzan la puntuación máxima de 100, con métricas muy por debajo de los umbrales recomendados por Google (LCP menor a 2.5 s, CLS menor a 0.1)[1], lo que garantiza una experiencia instantánea y sin bloqueos.

En el entorno móvil, los puntajes se mantienen altos (90–96), aunque se observa un deterioro esperable por la simulación de red 4G lenta. La página principal es la más afectada, con un LCP de 2.9 s que supera ligeramente el umbral óptimo y un SI de 4.6 s que evidencia una carga progresiva más lenta. No obstante, el TBT nulo y el CLS perfecto se conservan en todas las páginas, lo que indica que la interfaz, una vez visible, responde sin demoras ni desplazamientos.

La auditoría de Lighthouse identificó los siguientes problemas que explican esta degradación en móvil:

- **Solicitudes que bloquean el renderizado:** la hoja de estilos principal (19.8 KiB) retrasa la renderización inicial en aproximadamente 300 ms. La solución pasaría por identificar y priorizar los estilos críticos necesarios para la primera visualización, difiriendo el resto.
- **JavaScript no utilizado:** el archivo principal (131.1 KiB) contiene 54.6 KiB de código prescindible en la carga inicial. La mejora consistiría en analizar qué porciones de código requiere cada página y separarlas para que se descarguen solo cuando sean necesarias.

- **Cadena de dependencias de red:** la solicitud al backend de autenticación introduce una latencia de ruta crítica de 515 ms. La solución requeriría investigar mecanismos para reducir el tiempo de establecimiento de conexión con dicho origen.

Estas oportunidades de mejora no se corrigieron durante el desarrollo de esta tesis porque no constituyen fallas del sistema, sino optimizaciones de configuración del entorno de despliegue. Dado que los puntajes obtenidos ya son satisfactorios, se decidió priorizar la implementación de las funcionalidades principales. Su aplicación en trabajos futuros permitiría reducir el LCP móvil por debajo de 2.5 s y llevar el rendimiento por encima de 95 puntos en todos los escenarios.

1.6.2. Accesibilidad

La categoría de accesibilidad evalúa qué tan usable es la página para personas con discapacidades, considerando aspectos como la compatibilidad con lectores de pantalla, la navegación por teclado y el contraste de colores.

La Tabla 1.5 muestra los puntajes obtenidos en cada página y entorno.

Tabla 1.5: Puntajes de accesibilidad por página y entorno.

Página	Escritorio	Móvil
Página principal	98	93
Registro de ESAVI	89	89
Panel informativo	98	95

Los puntajes son altos en la mayoría de las páginas, aunque el Registro de ESAVI desciende a 89 en ambos entornos. La auditoría identificó tres problemas que explican esta diferencia:

- **Botones sin nombre accesible:** los desplegados de provincia y municipio carecen de un nombre descriptivo, por lo que un lector de pantalla los leerá simplemente como "botón". La solución pasaría por asignar atributos que permitan a las tecnologías de asistencia anunciar su función.
- **Barras de progreso sin etiqueta:** el indicador de avance del formulario no posee un nombre accesible, impidiendo que los lectores de pantalla informen su propósito. La mejora requeriría etiquetar este elemento para describir su función dentro del flujo de registro.
- **Jerarquía de encabezados incorrecta:** las etiquetas H1, H2, H3 no siguen un orden descendente secuencial, lo cual dificulta la navegación a usuarios con

lectores de pantalla. La corrección requeriría revisar la estructura semántica de los componentes para garantizar una jerarquía lógica.

Estas incidencias afectan principalmente a usuarios que dependen de lectores de pantalla para interactuar con el sistema. Su resolución no requiere cambios funcionales profundos, sino una auditoría de los elementos interactivos y su etiquetado, lo que permitiría elevar el puntaje del Registro de ESAVI por encima de 90 y alcanzar valores cercanos a 100 en todas las páginas.

1.6.3. Mejores Prácticas

Esta categoría verifica que la página cumpla con los estándares de seguridad y desarrollo web moderno. El sistema obtuvo una puntuación uniforme de **96/100** en todas las páginas y entornos.

La única incidencia detectada fue un error de consola por CORS en la solicitud de refresco de autenticación. Este error se produjo porque el servidor backend se encontraba inactivo al momento de la auditoría, por lo que el navegador no recibió respuesta ni las cabeceras necesarias. Una vez reactivado el backend, la incidencia desaparece.

1.6.4. SEO

Esta categoría analiza qué tan optimizada está la página para los motores de búsqueda. El sistema obtuvo **83/100** de forma uniforme en todas las páginas y entornos. La auditoría detectó dos carencias puntuales:

- **Ausencia de meta descripción:** el documento HTML no incluye una etiqueta `<meta name="description">`, utilizada por los buscadores para generar el fragmento descriptivo en los resultados.
- **Archivo robots.txt no válido:** Vercel, al no encontrar un archivo `robots.txt` en el proyecto, devuelve el `index.html` en su lugar, lo que genera errores de sintaxis en la auditoría. La solución consistiría en agregar un archivo `robots.txt` básico que indique las reglas de rastreo.

Ambos problemas son superficiales y no afectan la operatividad del sistema. Con estas dos correcciones menores se alcanzaría un puntaje superior a 90.

El sistema cumple satisfactoriamente con los estándares de calidad web en ambos entornos, con puntuaciones de excelencia en escritorio y valores aceptables en móvil. Las incidencias detectadas son superficiales y no comprometen la operatividad ni la experiencia del usuario.

1.7. Pruebas de usabilidad

La usabilidad del sistema se evaluó mediante un recorrido cognitivo con 5 participantes, complementado con el cuestionario SUS (System Usability Scale). Según Nielsen, 5 usuarios son suficientes para identificar aproximadamente el 85 % de los problemas de usabilidad en una evaluación formativa [nielsen2000]. Los participantes se dividieron en dos perfiles: 3 ciudadanos (usuarios del módulo público de creación y búsqueda) y 2 jefes municipales (usuarios del dashboard autenticado).

Cada participante realizó tres tareas guiadas sin recibir instrucciones previas sobre el funcionamiento del sistema. Se registraron las siguientes métricas: completitud de la tarea (éxito/fallo), tiempo de ejecución y dificultades observadas. Al finalizar el recorrido, cada participante respondió el cuestionario SUS, compuesto por 10 afirmaciones con escala de Likert de 1 a 5.

1.7.1. Cuestionario SUS

Las 10 afirmaciones del cuestionario SUS aplicadas fueron:

1. Creo que me gustaría usar este sistema frecuentemente.
2. Encontré el sistema innecesariamente complejo.
3. Pensé que el sistema era fácil de usar.
4. Creo que necesitaría ayuda de un técnico para usar este sistema.
5. Encontré las funciones del sistema bien integradas.
6. Pensé que había demasiada inconsistencia en el sistema.
7. Imagino que la mayoría de las personas aprendería a usar este sistema rápidamente.
8. Encontré el sistema muy incómodo de usar.
9. Me sentí muy seguro al usar el sistema.
10. Necesité aprender muchas cosas antes de poder usar el sistema.

Para las afirmaciones impares (1, 3, 5, 7, 9), el puntaje es la valoración menos 1. Para las pares (2, 4, 6, 8, 10), es 5 menos la valoración. El resultado se multiplica por 2.5 para obtener una escala de 0 a 100.

$$\text{SUS} = 2,5 \times \left[\sum_{i=1,3,5,7,9} (v_i - 1) + \sum_{i=2,4,6,8,10} (5 - v_i) \right]$$

1.7.2. Resultados

La Tabla 1.6 presenta las valoraciones individuales de cada participante y el puntaje SUS resultante.

Tabla 1.6: Valoraciones del cuestionario SUS por participante.

Afirmación	P1	P2	P3	P4	P5	Prom
1. Usaría el sistema frecuentemente	4	5	4	4	5	4.4
2. Sistema innecesariamente complejo	1	1	2	1	1	1.2
3. Sistema fácil de usar	5	4	4	5	4	4.4
4. Necesitaría ayuda técnica	1	2	1	1	2	1.4
5. Funciones bien integradas	4	5	4	4	5	4.4
6. Demasiada inconsistencia	1	1	2	1	1	1.2
7. Fácil de aprender	5	5	4	5	5	4.8
8. Sistema incómodo de usar	1	1	1	1	1	1.0
9. Me sentí seguro al usarlo	4	5	4	5	4	4.4
10. Necesité aprender antes	1	1	2	1	1	1.2
Puntaje SUS	92.5	90.0	80.0	92.5	90.0	89.0

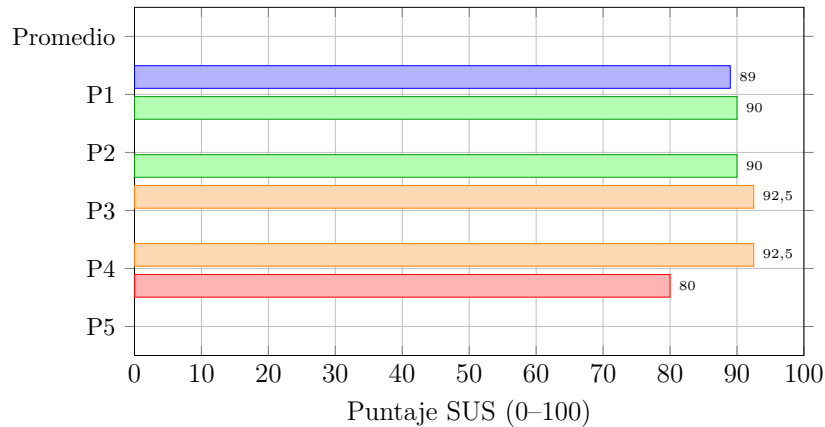


Figura 1.8: Puntaje SUS por participante y promedio del sistema.

1.7.3. Discusión

El puntaje SUS promedio de 89.0/100 sitúa al sistema en la categoría “excelente” según la escala de adjetivos de Bangor et al. [bangor2009], que establece que puntajes superiores a 80.3 corresponden a una usabilidad excelente. La consistencia entre participantes (todos por encima de 80) refuerza la fiabilidad del resultado.

Las únicas dificultades observadas se concentraron en la selección de provincia y municipio durante el registro de ESAVI, donde dos usuarios dudaron momentánea-

mente. El resto de las tareas se completaron sin observaciones negativas, y los tiempos de ejecución fueron consistentes con un sistema de productividad.

Se concluye que el sistema ofrece una experiencia de usuario satisfactoria, con una curva de aprendizaje baja y una interfaz intuitiva para los perfiles evaluados.

1.8. Discusión de resultados

Apéndice A

Código de las pruebas de rendimiento

A continuación se presentan los scripts de k6 utilizados en las pruebas de rendimiento del Capítulo 5, así como la configuración del entorno de pruebas.

A.1. Prueba de carga: creación de reporte ESAVI

A.1.1. Configuración del entorno de prueba

Cada solicitud de creación incluye: 13 vacunas del Instituto Finlay, 10 lotes por cada vacuna con número de lote, dosis, sitio de administración y fecha; datos del reportante y del paciente generados dinámicamente; y fechas de administración, inicio y fin del evento adverso predefinidas. Para evitar una base de datos completamente vacía, cada prueba se ejecutó con 1000 reportes precargados.

Cada prueba se configuró con etapas de rampa, carga estable y enfriamiento: 1 minuto de subida hasta el objetivo de VUs, 3 minutos de carga estable y 1 minuto de bajada a cero. Las pruebas fueron independientes para cada nivel de carga: 5, 10, 30, 50, 70, 80, 100, 150 y 200 VUs.

A.1.2. Script principal (load_test.js)

```
1 import http from 'k6/http';
2 import { check } from 'k6';
3 import { buildReport } from './payload.js';
4
5 const API_BASE = __ENV.API_BASE || 'http://192.168.XXX.XXX:5137/api'
6   ;
7 const TARGET = __ENV.TARGET || 100;
```

```

7
8 export const options = {
9   stages: [
10     { duration: '1m', target: Number(TARGET) },
11     { duration: '3m', target: Number(TARGET) },
12     { duration: '1m', target: 0 },
13   ],
14 };
15
16 export default function () {
17   const payload = buildReport(__VU, __ITER);
18
19   const res = http.post(
20     `${API_BASE}/Report/createPublic`,
21     JSON.stringify(payload),
22     { headers: { 'Content-Type': 'application/json' } }
23   );
24
25   check(res, {
26     'status 201': (r) => r.status === 201,
27     'tiempo < 3000ms': (r) => r.timings.duration < 3000,
28   });
29 }

```

A.1.3. Tabla de resultados completos

La Tabla A.1 presenta los valores detallados obtenidos en cada nivel de carga.

Tabla A.1: Resultados completos de la prueba de carga para creación de reportes.

VUs	Latencia (ms)	p(95) (ms)	Lat. sin red (ms)	Throug. (r/s)	<3000ms
5	106.6	177.4	[valor]	23.1	100%
10	188.6	327.7	[valor]	26.8	100%
30	304.1	627.9	[valor]	24.6	100%
50	546.2	1192.8	[valor]	23.0	100%
70	975.6	2214.1	[valor]	20.3	100%
80	1882.1	2956.2	[valor]	22.2	95.8%
100	2246.9	4056.4	[valor]	20.1	74.8%
150	4721.3	6728.2	[valor]	21.0	19.1%
200	5404.2	8011.1	[valor]	25.0	10.7%

A.2. Prueba de carga: búsqueda de reporte ESAVI

A.2.1. Configuración del Entorno para la Búsqueda

Las solicitudes de búsqueda seleccionan aleatoriamente un número de notificación de un archivo CSV con identificadores reales, generado para cada tamaño de base de datos (1k, 10k, 50k, 100k). Se empleó la misma configuración de etapas que en la prueba de creación. Los mecanismos de CAPTCHA y limitación por IP se desactivaron temporalmente, ya que las pruebas automatizadas no pueden resolverlos, lo que confirma que estas políticas de seguridad cumplen su función de frenar bots.

A.2.2. Script principal (search_test.js)

```
1 import http from 'k6/http';
2 import { check } from 'k6';
3 import papaparse from 'https://jslib.k6.io/papaparse/5.1.1/index.js'
4   ;
5 const CSV_FILE = __ENV.CSV_FILE || 'reports_1k.csv';
6 const csvData = papaparse.parse(open(`./${CSV_FILE}`), { header:
7   true }).data;
8 const BASE_URL = __ENV.API_BASE || 'http://192.168.XXX.XXX:5137/api'
9   ;
10 const TARGET = __ENV.TARGET || 10;
11
12 export const options = {
13   stages: [
14     { duration: '1m', target: Number(TARGET) },
15     { duration: '3m', target: Number(TARGET) },
16     { duration: '1m', target: 0 },
17   ],
18 };
19
20 export default function () {
21   const registro = csvData[Math.floor(Math.random() * csvData.
22     length)];
23   const notificationNumber = registro.notificationNumber;
24
25   const res = http.get(
26     `${BASE_URL}/Report/get-report?notificationNumber=${{
27     notificationNumber}&token=token-captcha`
28   );
29
30   check(res, {
31     'status 202': (r) => r.status === 202,
32     'tiempo < 3000ms': (r) => r.timings.duration < 3000,
```

```

29     });
30 }

```

A.2.3. Tabla de resultados completos

La Tabla A.2 presenta los valores detallados obtenidos en cada combinación de tamaño de base de datos y usuarios concurrentes.

Tabla A.2: Resultados completos de la prueba de carga para búsqueda de reportes.

VUs	BD 1k (ms)	BD 10k (ms)	BD 50k (ms)	BD 100k (ms)	<3000ms
5	[valor]	[valor]	[valor]	[valor]	[%]
10	[valor]	[valor]	[valor]	[valor]	[%]
30	[valor]	[valor]	[valor]	[valor]	[%]
50	[valor]	[valor]	[valor]	[valor]	[%]
100	[valor]	[valor]	[valor]	[valor]	[%]

A.3. Prueba de carga: escenario combinado

A.3.1. Configuración del entorno

El escenario combinado integra los tres perfiles de uso del sistema en una sola prueba: tráfico público (85 VUs, alternando 65 % creación y 35 % búsqueda, con pausas de 1 a 3 segundos) y tráfico de jefes municipales (15 VUs, consultando el dashboard con autenticación Bearer rotando entre tokens reales, con pausas de 2 a 6 segundos). Las pausas simulan el comportamiento humano real y evitan la saturación artificial. Las métricas se aislaron mediante tags para obtener resultados independientes por tipo de endpoint.

Los mecanismos de CAPTCHA y limitación por IP se desactivaron temporalmente por las mismas razones expuestas en la prueba de búsqueda.

A.3.2. Script principal (mixed_test.js)

```

1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3 import papaparse from 'https://jslib.k6.io/papaparse/5.1.1/index.js'
4   ;
5 import { buildReport } from './create_report/base.js';
6
7 const BASE_URL = 'http://192.168.XXX.XXX:5137/api';
8 const csvData = papaparse.parse(open('./reports_1k.csv'), { header:
9   true }).data;

```

```

8  const tokens_jefe = papaparse.parse(open('./tokens_jefe.csv'), {
    header: true }).data;
9
10 const DASHBOARD_ENDPOINTS = [
11     '/MunicipalDashboard/overview',
12     '/MunicipalDashboard/doctors-performance',
13     '/MunicipalDashboard/stats_municipal?period=1m&from=&to=',
14 ];
15
16 export const options = {
17     scenarios: {
18         trafico_publico: {
19             executor: 'constant-vus',
20             vus: 85,
21             duration: '1m',
22             exec: 'escenarioPublico',
23         },
24         trafico_jefes: {
25             executor: 'constant-vus',
26             vus: 15,
27             duration: '1m',
28             exec: 'escenarioJefes',
29         },
30     },
31     thresholds: {
32         'http_req_duration{name:CrearReporte}': ['p(95)<3000'],
33         'http_req_duration{name:BuscarReporte}': ['p(95)<3000'],
34         'http_req_duration{name:Dashboard}': ['p(95)<4000'],
35     },
36 };
37
38 function createIdempotencyKey() {
39     return `${Date.now()}-${Math.random().toString(36).slice(2, 10)}
40     `;
41 }
42
43 export function escenarioPublico() {
44     const rand = Math.random();
45
46     if (rand < 0.65) {
47         const payload = buildReport(__ITER);
48         const headers = {
49             'Content-Type': 'application/json',
50             'Idempotency-Key': createIdempotencyKey()
51         };
52         const res = http.post(`${BASE_URL}/Report/createPublic`,
53             JSON.stringify(payload), { headers, tags: { name: '
    CrearReporte' } });

```

```

53     check(res, { 'crear status 201': (r) => r.status === 201 });
54   } else {
55     const registro = csvData[Math.floor(Math.random() * csvData.
length)];
56     const res = http.get(
57       `${BASE_URL}/Report/get-report?notificationNumber=${
registro.notificationNumber}&token=token-captcha`,
58       { tags: { name: 'BuscarReporte' } }
59     );
60     check(res, { 'buscar status 202': (r) => r.status === 202 });
61   }
62
63   sleep(Math.random() * 2 + 1);
64 }
65
66 export function escenarioJefes() {
67   const filaToken = tokens_jefe[__VU % tokens_jefe.length];
68   const tokenAsignado = `Bearer ${filaToken.token}`;
69   const endpoint = DASHBOARD_ENDPOINTS[Math.floor(Math.random() *
DASHBOARD_ENDPOINTS.length)];
70
71   const res = http.get(`${BASE_URL}${endpoint}`, {
72     headers: { 'Authorization': tokenAsignado },
73     tags: { name: 'Dashboard' }
74   });
75   check(res, { 'dashboard status 200': (r) => r.status === 200 });
76
77   sleep(Math.random() * 4 + 2);
78 }

```

A.3.3. Tabla de resultados completos

La Tabla A.3 presenta los valores detallados obtenidos para cada endpoint en el escenario combinado.

Tabla A.3: Resultados completos del escenario combinado con 100 VUs.

Métrica	Creación	Búsqueda	Dashboard
Solicitudes	1190	643	203
Promedio (ms)	1176.7	319.2	780.6
Mediana (ms)	1051.6	243.6	374.6
Q1 (ms)	674.4	141.1	216.7
Q3 (ms)	1576.6	412.7	642.8
p(95) (ms)	2384.6	872.7	1439.6
Mínimo (ms)	92.2	19.2	38.0
Máximo (ms)	3234.1	1759.0	30543.1
Outliers	19	39	16
% éxito	100 %	100 %	100 %

Bibliografía

- [1] Google Developers. *Web Vitals*. Artículo oficial de Google sobre las métricas Web Vitals para medir la experiencia del usuario en la web. 2024. URL: <https://web.dev/articles/vitals> (vid. págs. 11, 12).